

HW 2-B

Course: CSCE 703

UIN: 937006978

Email: srijapentyala@tamu.edu

GitHub: <https://github.com/srijapentyala/hw2b-juice-shop-login>

Part 1: Secure Feature Design

I ran Juice Shop locally. The table records three bugs I reproduced and the registration control that stops each one.

Vulnerability	Exploit Scenario	Security Mitigation
SQL Injection	I typed ' OR 1=1-- and a junk password. Juice Shop signed me in as admin@juice-sh.op and awarded Login Admin. The email is concatenated into the query, so the injected clause is treated as SQL.	A registration insert must use a parameterized query. The email stays a bound value, so it cannot add an administrator row or change the statement.
Cross-Site Scripting	Search writes the query into the page as HTML. My term rendered as a yellow XSS heading instead of text. A tag in a registered username would run the same way.	Encode the username on output and assign it with <code>textContent</code> . Send <code>Content-Security-Policy: script-src 'self'</code> so a tag is shown and does not run.
Weak Password Storage	The administrator password is admin123, and the login token includes its MD5 hash. Password Strength solved only when I used that password, not injection.	Reject common passwords. Store a bcrypt hash with a unique salt and cost 12, and never return the hash.

How These Measures Prevent Attacks

A parameterized query sends the email as data, so ' OR 1=1-- cannot change the statement.

Encoding the username, and refusing inline script, means a tag in the name is displayed as text.

bcrypt at cost 12 is slow and uses a new salt for every password, so the MD5 in the Juice Shop token cannot be reused. Registration should run this check on the server, not only in the browser.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
bcrypt.checkpw(password.encode(), stored_hash)
```

Figure 1 keeps the injected email in the box and shows both challenge banners. Figure 2 is the same kind of mistake in search: the term is drawn as a heading, so a username must be encoded before a registration page displays it.

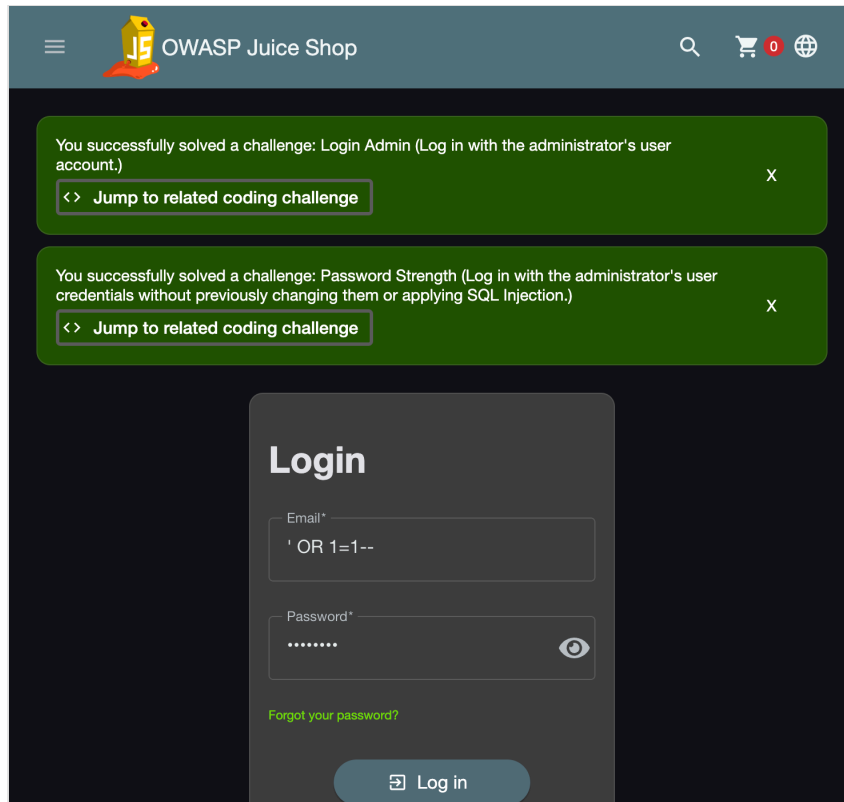


Figure 1. The injected email is still in the form. Juice Shop awarded Login Admin for that login, and Password Strength for admin123.

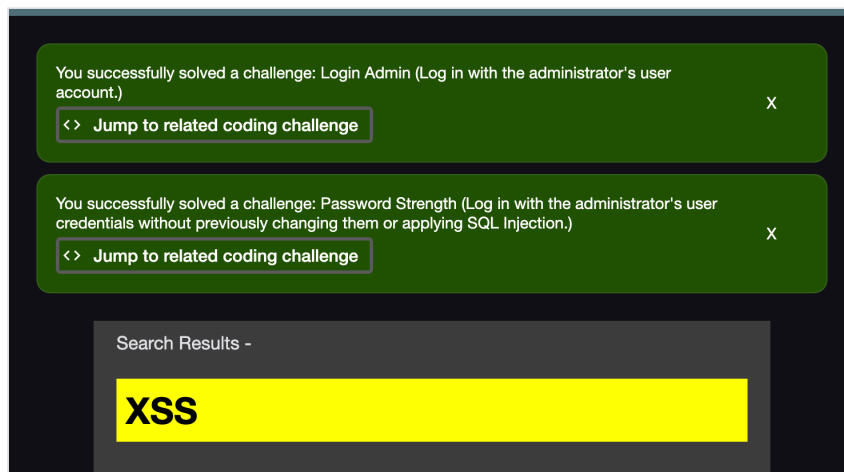


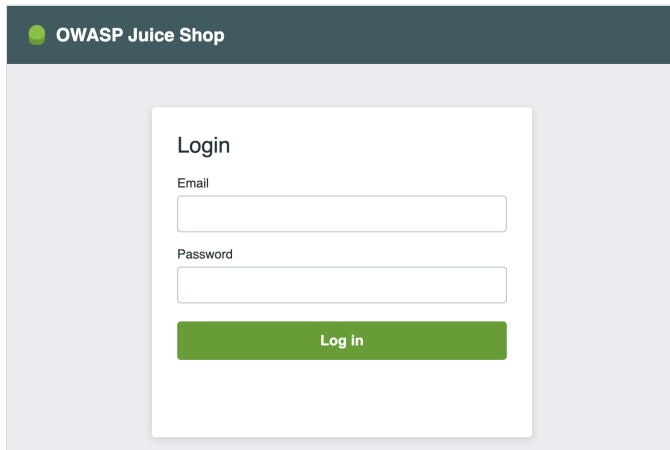
Figure 2. The search term was parsed as HTML. The tags never appear as text.

Part 2: Simple Front-End Form

GitHub: <https://github.com/srijapentyala/hw2b-juice-shop-login>

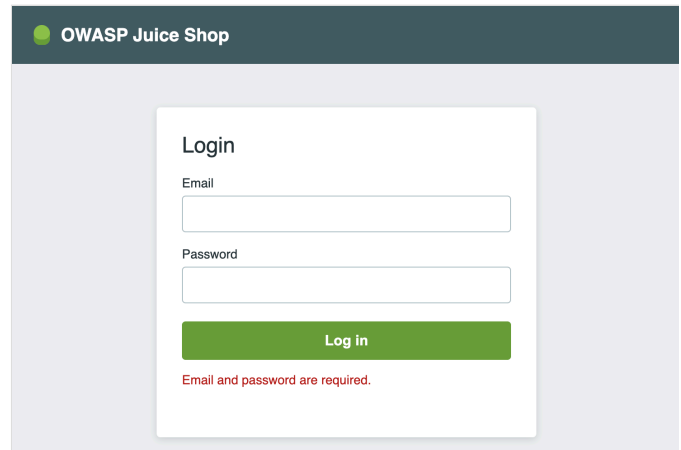
I built an HTML and JavaScript login form with Juice Shop's two fields, email and password. The form uses novalidate, so login.js reports the error instead of the browser popup. It blocks an empty submit, an email with no "@" or an "@" at either end, and a password shorter than 8 characters. Those three errors stay in the browser and no request is sent. A form that passes is posted as JSON to POST /login. server.py repeats the same checks and answers HTTP 400 when the format is bad. The only account is demo@juice-sh.op / JuiceShop1. The password is not stored. The server keeps a salt and a PBKDF2-HMAC-SHA256 hash of 200,000 rounds and compares it with hmac.compare_digest. A wrong password and an unknown email both get HTTP 401 and the sentence "Invalid email or password." Nothing is

placed in a SQL string. A body over 4096 bytes is rejected, and each response sends `X-Content-Type-Options: nosniff`. Run `python3 server.py` and open `http://127.0.0.1:8080`. The status line is plain text. The line under it writes the email with `innerHTML`, which Part 3 exploits.



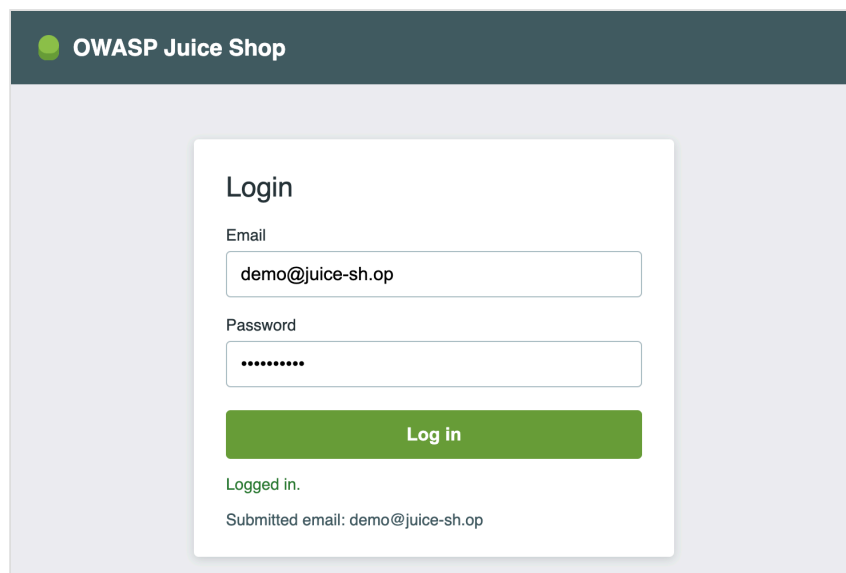
The screenshot shows the OWASP Juice Shop login page. It has a dark blue header with the OWASP Juice Shop logo. The main content area is light gray. In the center is a white login form with the title "Login". It contains two input fields: "Email" and "Password". Below the fields is a green "Log in" button.

Figure 3. Login form before submission.



The screenshot shows the OWASP Juice Shop login page after an empty submission. The form is the same as in Figure 3, but now there is a red error message below the "Log in" button that says "Email and password are required."

Figure 4. The browser rejects an empty submission.



The screenshot shows the OWASP Juice Shop login page after a successful login. The form is the same as in Figure 3, but now the "Email" field contains the text "demo@juice-sh.op" and the "Password" field contains a series of dots. Below the "Log in" button, there is a green message that says "Logged in." and a line of text that says "Submitted email: demo@juice-sh.op".

Figure 5. The demo account passes both the browser check and the server check.

Part 3: Exploiting a Vulnerability

I started `python3 server.py`, opened `http://127.0.0.1:8080`, and repeated the Juice Shop attacks on this form.

1. SQL injection. I typed `' OR 1=1--@juice-sh.op` in Email and `password1` in Password, then clicked Log in. The `@` lets the format check pass, so the request reaches the server. The answer was "Invalid email or password." The quote and the OR clause stayed characters in an email address, because the server never builds a SQL string. Login failed.

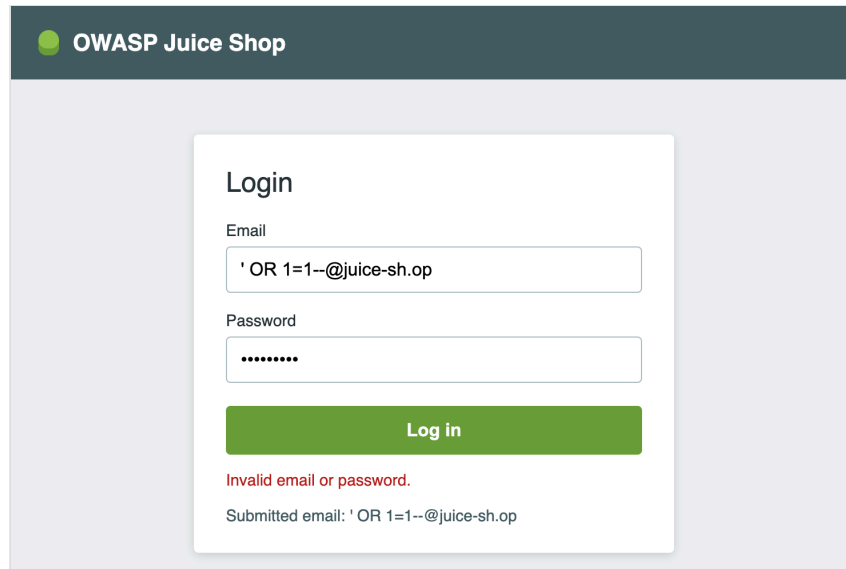


Figure 6. The SQL fragment is stored as an email address. Login fails.

2. XSS, which succeeds on this repository. `login.js` writes the submitted email with `innerHTML`. A `<script>` tag inserted that way does not run, so I used an image whose load fails. I typed `<img src=x onerror='document.getElementById('message').textContent='JavaScript executed''>@a.co` and password1, then clicked Log in. The picture did not load, `onerror` ran, and the status line became “JavaScript executed.” The login itself still failed, so the script ran in the page even though authentication did not succeed.

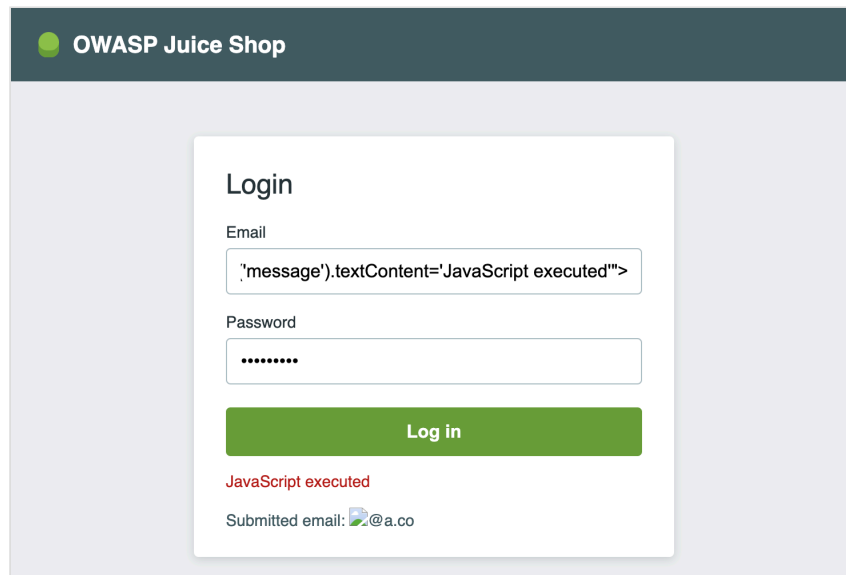


Figure 7. The status line was written by the image handler, not by the login code.

The repository still contains that `innerHTML` line so Figure 7 can be repeated from the public code. The fix is to assign the email with `textContent` and to send `Content-Security-Policy: script-src 'self'`. The same payload is then shown as text, and the handler cannot run.